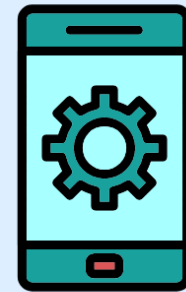


Reverse engineering

Basics

Ruxandra F. Olimid

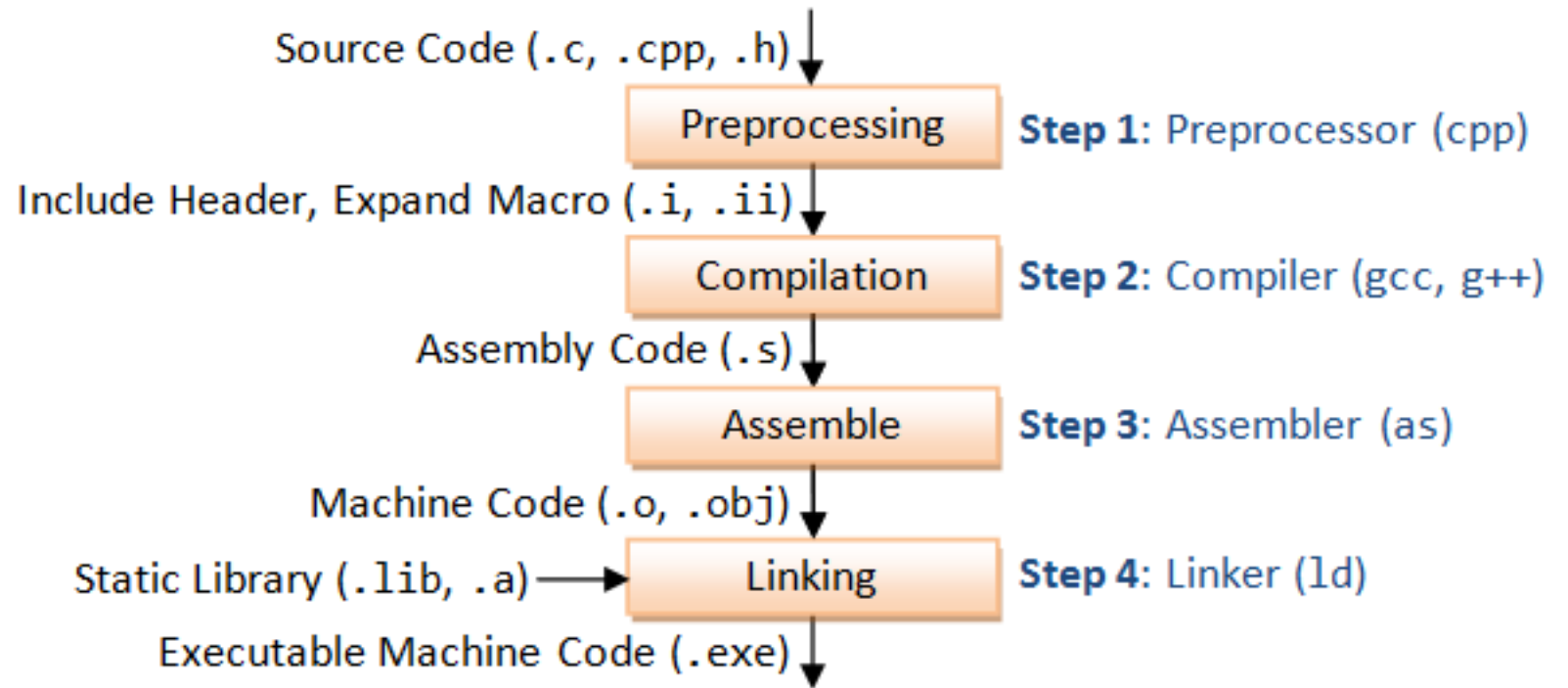
February 25, 2026



Agenda

- From the source code to the binary code/executable
- What in Reverse Engineering?
- Assambly x86

From source code to executable - GCC



GCC compiles a C/C++ program into executable in 4 steps as shown in the above diagram. For example, a "gcc -o hello.exe hello.c" is carried out as follows:

1. Pre-processing: via the GNU C Preprocessor (cpp.exe), which includes the headers (#include) and expands the macros (#define).

```
> cpp hello.c > hello.i
```

The resultant intermediate file "hello.i" contains the expanded source code.

2. Compilation: The compiler compiles the pre-processed source code into assembly code for a specific processor.

```
> gcc -S hello.i
```

The -s option specifies to produce assembly code, instead of object code. The resultant assembly file is "hello.s".

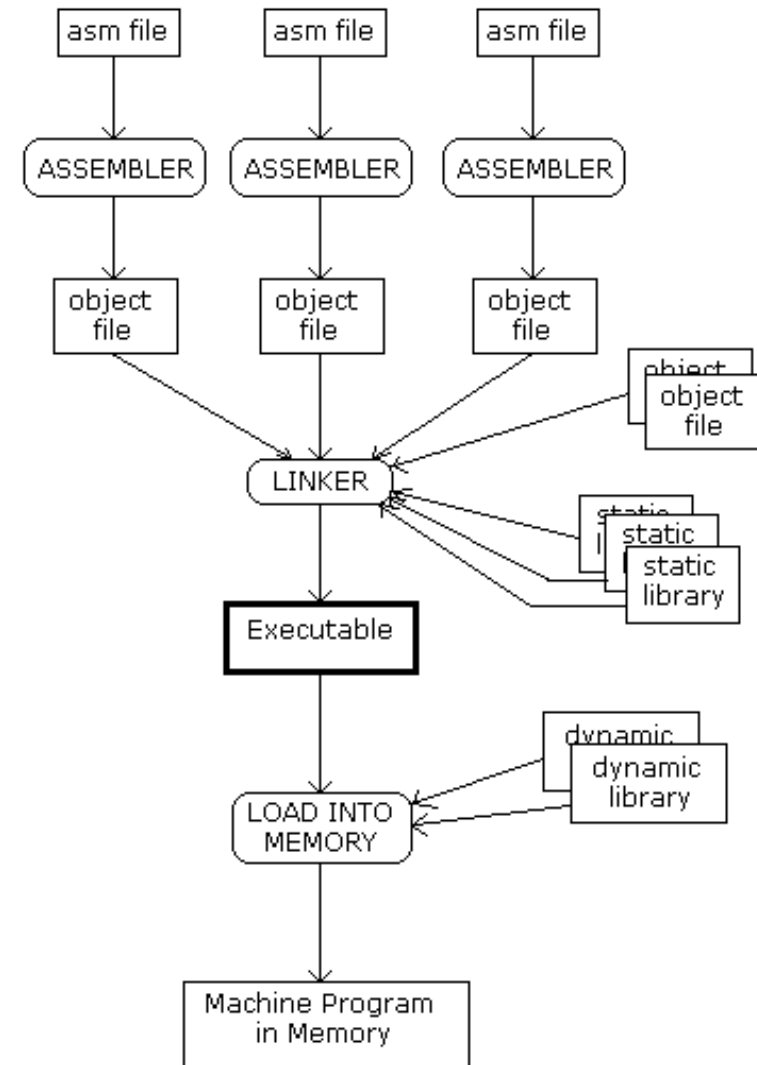
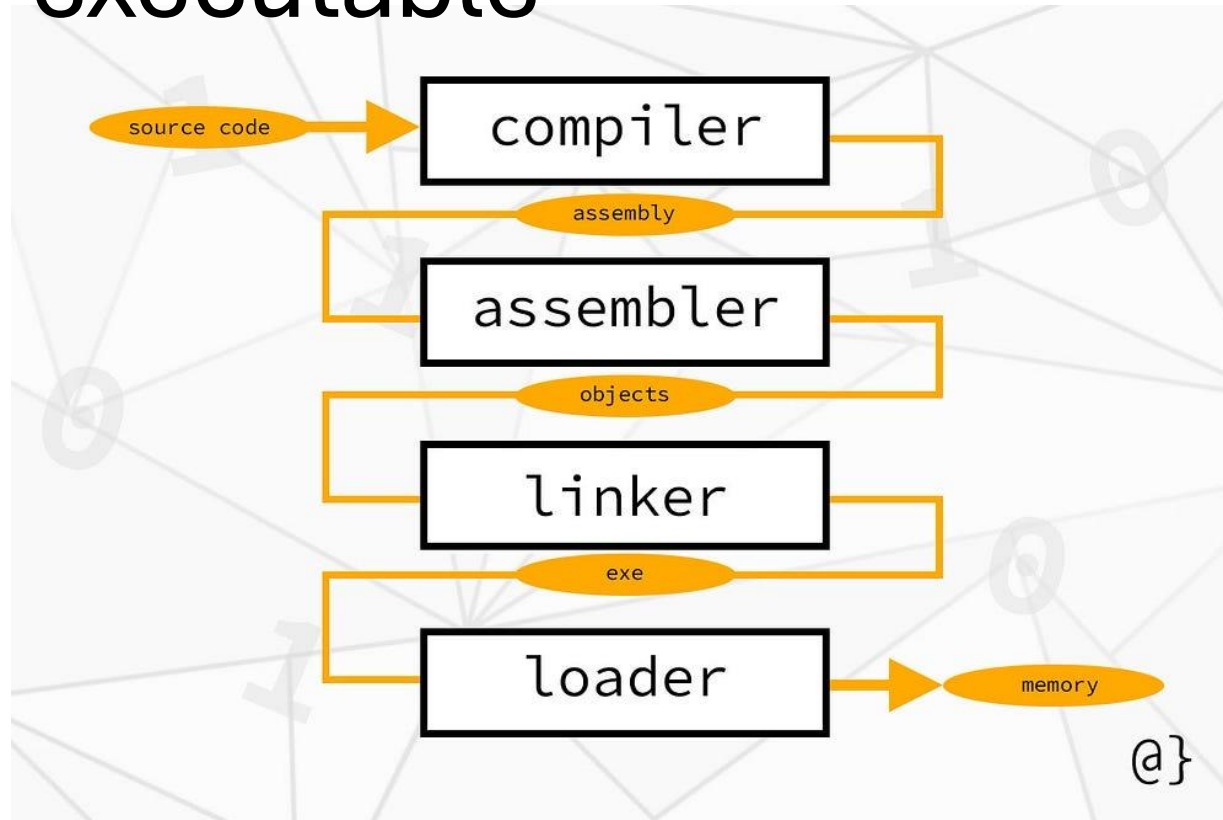
3. Assembly: The assembler (as.exe) converts the assembly code into machine code in the object file "hello.o".

```
> as -o hello.o hello.s
```

4. Linker: Finally, the linker (ld.exe) links the object code with the library code to produce an executable file "hello.exe".

```
> ld -o hello.exe hello.o ...libraries...
```

From source code to executable



He, X., Wang, S., Xing, Y., Feng, P., Wang, H., Li, Q., ... & Sun, K. (2022, October). Binprov: Binary code provenance identification without disassembly. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses* (pp. 350-363).

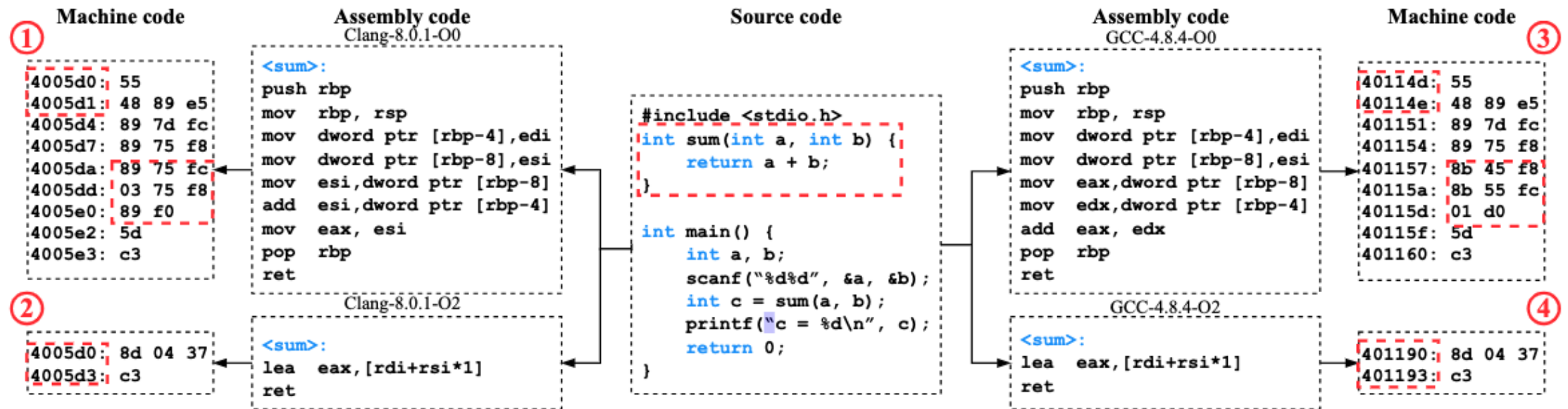


Figure 1: Four cases from source code to binary code under different compilers and optimization (opt) levels. Case 1 is compiled with 00 under Clang 8.0.1; Case 2 is compiled with 02 under Clang 8.0.1; Case 3 is compiled with 00 under GCC 4.8.4; Case 4 is compiled with 02 under GCC 4.8.4.

What is Reverse Engineering (RE)?

- Examples - Not only software RE (hardware, protocols, medicine, food, etc.)
- Goals, benefits of using RE (malware, identify weaknesses,),
- Stages of RE
- Tools (Ghidra, IDA Pro,)
- Ethical & legal aspects

What is Reverse Engineering?

- Zenarmor. Reverse Engineering in Cybersecurity

<https://www.zenarmor.com/docs/network-security-tutorials/what-is-reverse-engineering>

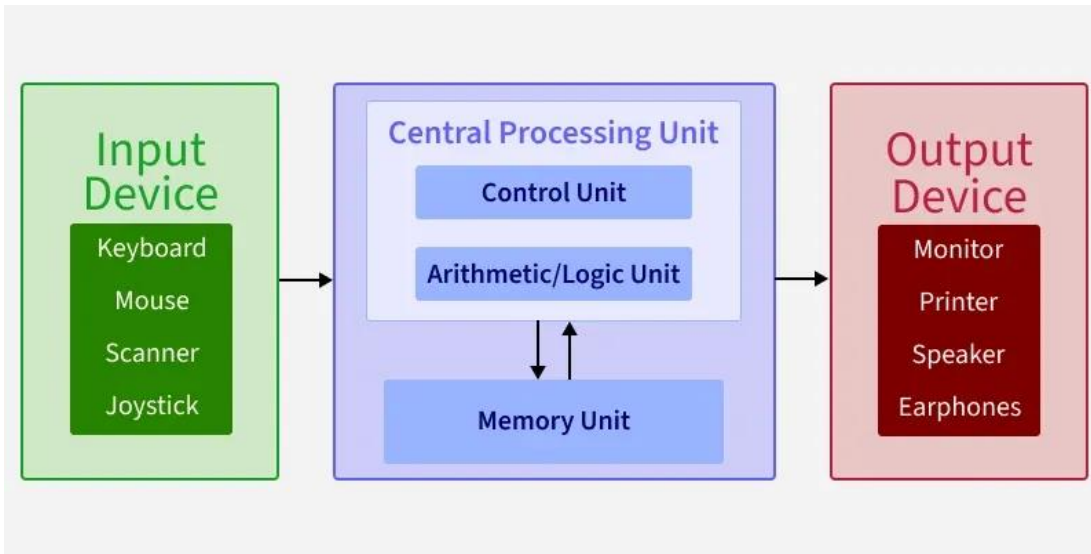
- Geeksforgeeks. What is Reverse Engineering Technique in Cybersecurity?

<https://www.geeksforgeeks.org/computer-networks/what-is-reverse-engineering-technique-in-cybersecurity/>

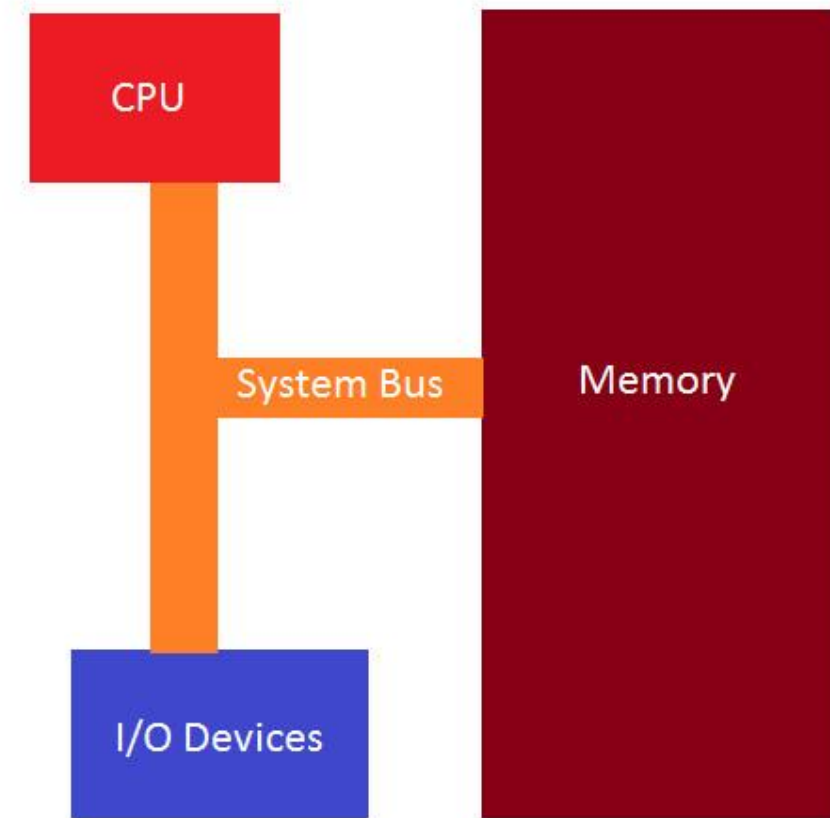
Remember

- Binary/hexadecimal system
- Byte (8 bits), word (2 bytes), double-word (4 bytes)
- 32-bit vs 64-bit architecture
- Little-endian vs big-endian
- Different ways to address memory
- ...

Basic Architecture



Source: <https://www.geeksforgeeks.org/computer-organization-architecture/computer-organization-von-neumann-architecture/>



Source: <https://0xinfection.xyz/reversing/pages/part-9-x86-basic-architecture.html>

CPU



Source: <https://0xinfection.xyz/reversing/pages/part-9-x86-basic-architecture.html>

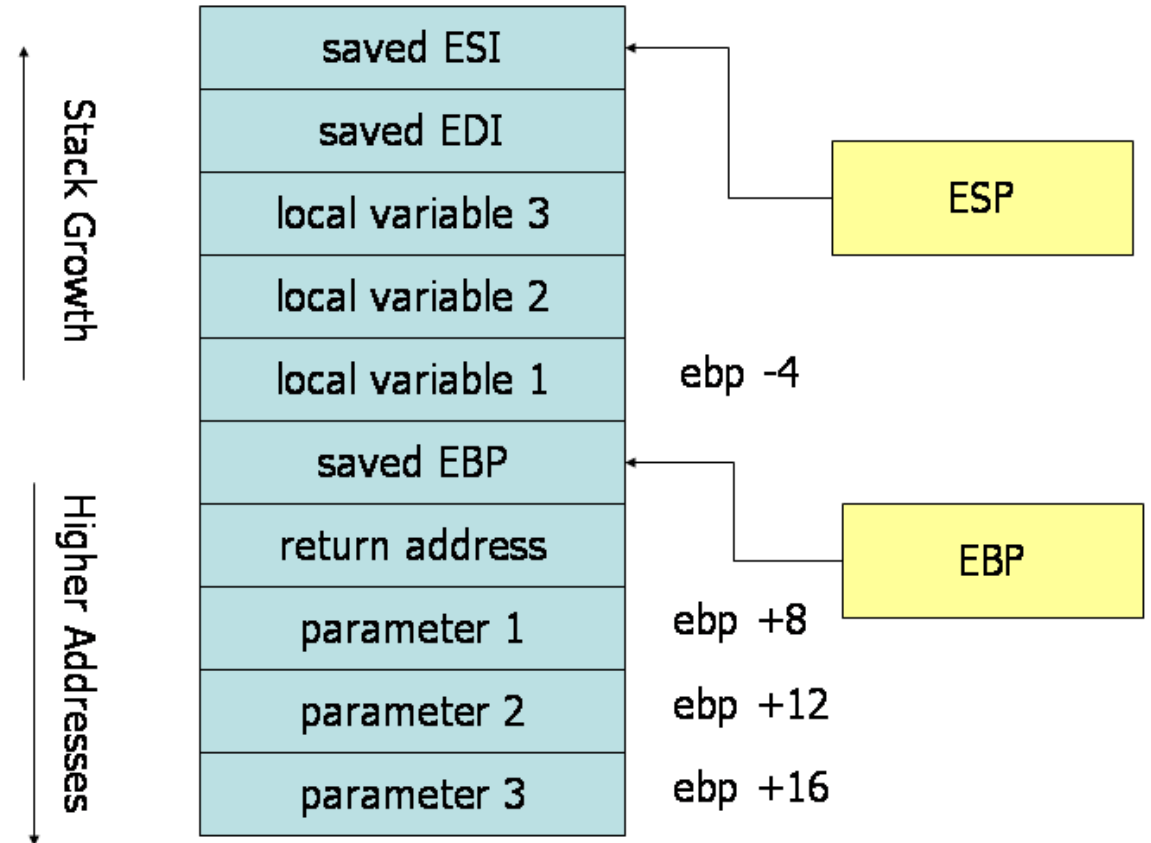
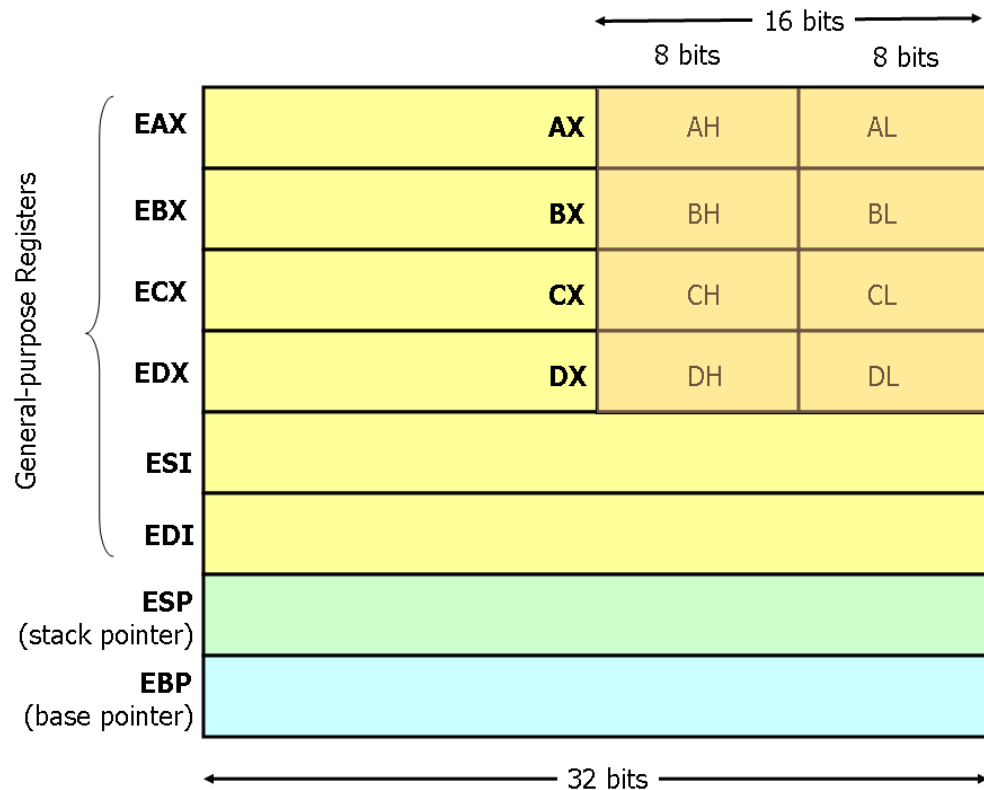
- **CU (Control Unit):** sends control signals, controls inputs and outputs operations, decodes instructions, fetches the instructions to/from memory for execution
- **Execution Unit/ALU (Arithmetic and Logic Unit):** executes the instructions, handles the calculations, performs arithmetic and logic operations
- **Registers:** fast memory located on the CPU

Flags registers: indicate special events

Instruction Pointer (IP) Register

- Points to the next instruction to be executed
- Example of IP hack: <https://0xinfection.xyz/reversing/pages/part-12-instruction-pointer-register.html>

Assembly x86



General-purpose Registers

Source: <https://www.c-jump.com/CIS77/ASM/Memory/lecture.html>

EAX: Main register used in arithmetic calculations. Also known as accumulator, as it holds results of arithmetic operations and function return values.

EBX: The Base Register. Pointer to data in the DS segment. Used to store the base address of the program.

ECX: The Counter register is often used to hold a value representing the number of times a process is to be repeated. Used for loop and string operations.

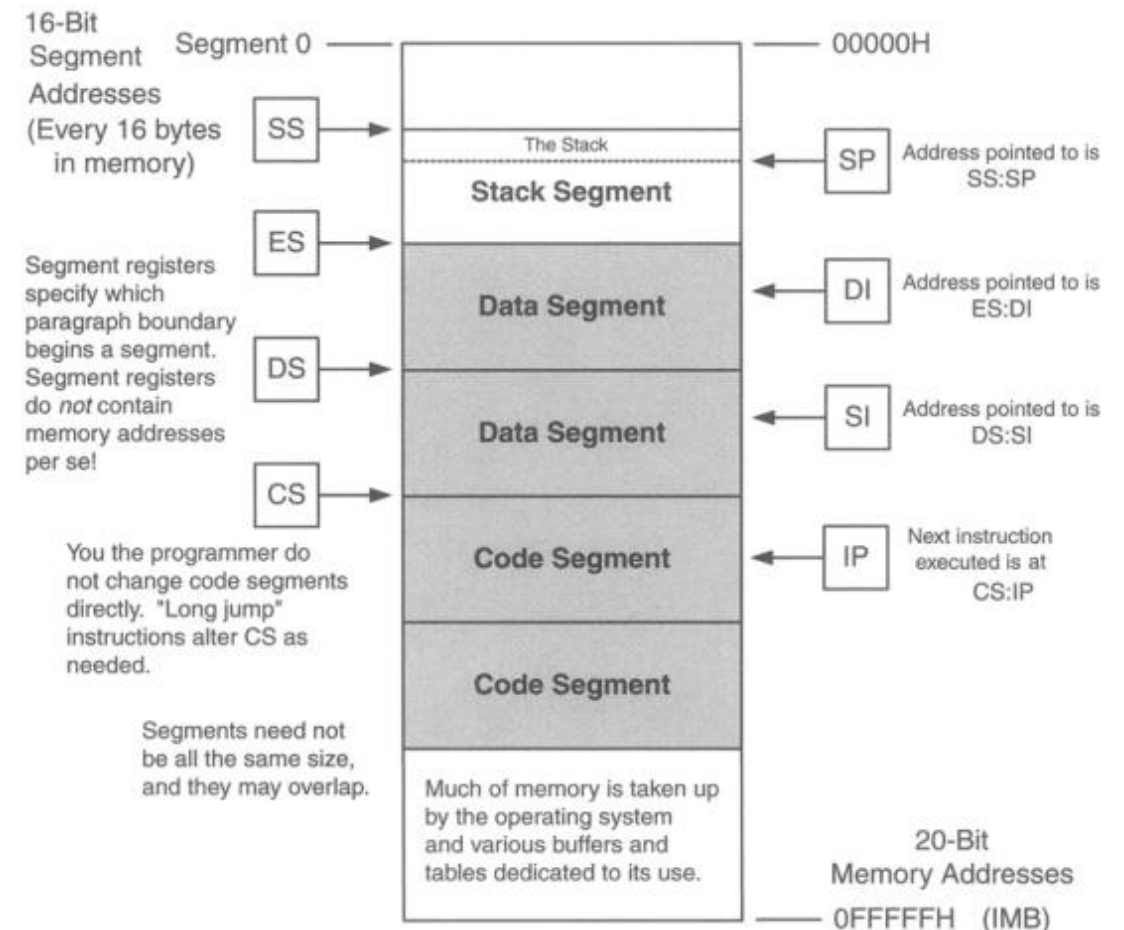
EDX: A general purpose register. Additionally used for I/O operations. In addition will extend EAX to 64-bits.

ESI: Source Index register. Pointer to data in the segment pointed to by the DS register. Used as an offset address in string and array operations. It holds the address from where to read data.

EDI: Destination Index register. Pointer to data (or destination) in the segment pointed to by the ES register. Used as an offset address in string and array operations. It holds the implied write address of all string operations.

EBP: Base Pointer. Pointer to data on the stack (in the SS segment). It points to the bottom of the current stack frame. It is used to reference local variables.

ESP: Stack Pointer (in the SS segment). It points to the top of the current stack frame. It is used to reference local variables.



Source: <https://0xinfection.xyz/reversing/pages/part-10-general-purpose-registers.html>

Assembly x86

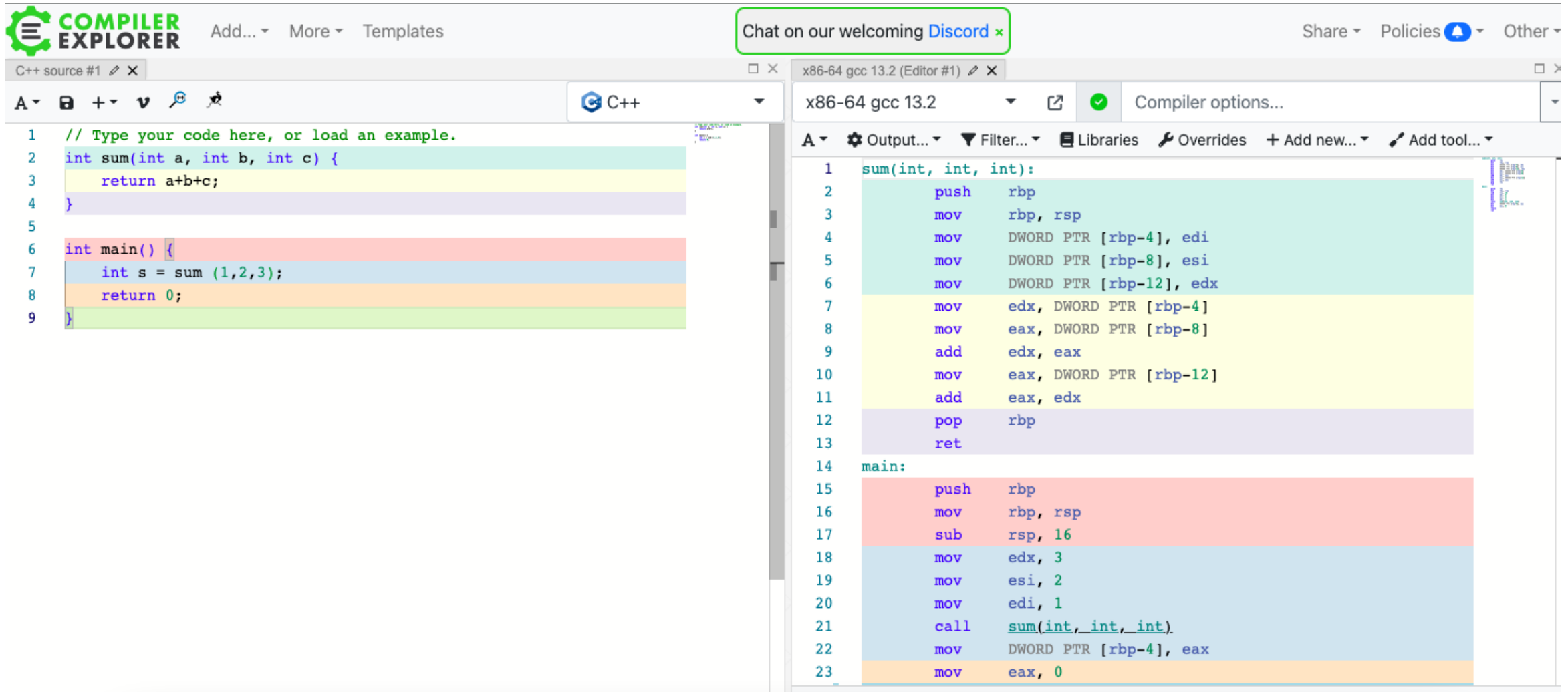
- Registers
 - General purpose
 - Special purpose (segment registers, control registers, flags)
- Instructions
 - ALU operations (add, sub, not, shl, shr, ...)
 - Data movement (mov, push, pop, ...)
 - Control flow (j, cmp ...)

Assembly x86

```
int sum(int a, int b, int c) {  
    return a+b+c;  
}
```

```
int main() {  
    int s = sum (1,2,3);  
    return 0;  
}
```

Assembly x86



The image shows the Compiler Explorer interface. On the left, the C++ source code is displayed in a file named 'C++ source #1'. It contains a simple function `sum` and a `main` function that calls it. On the right, the assembly output for 'x86-64 gcc 13.2' is shown. The assembly is color-coded to match the source code: light blue for the `sum` function, light red for the `main` function, and light orange for the `return` statement. The assembly shows stack frame setup, argument passing, function calls, and return value handling.

COMPILER EXPLORER Add... More Templates Chat on our welcoming [Discord](#) Share Policies Other

C++ source #1 C++

```
1 // Type your code here, or load an example.
2 int sum(int a, int b, int c) {
3     return a+b+c;
4 }
5
6 int main() {
7     int s = sum(1,2,3);
8     return 0;
9 }
```

x86-64 gcc 13.2 (Editor #1) x86-64 gcc 13.2 Compiler options...

Output... Filter... Libraries Overrides + Add new... + Add tool...

```
1 sum(int, int, int):
2     push    rbp
3     mov     rbp, rsp
4     mov     DWORD PTR [rbp-4], edi
5     mov     DWORD PTR [rbp-8], esi
6     mov     DWORD PTR [rbp-12], edx
7     mov     edx, DWORD PTR [rbp-4]
8     mov     eax, DWORD PTR [rbp-8]
9     add     edx, eax
10    mov     eax, DWORD PTR [rbp-12]
11    add     eax, edx
12    pop     rbp
13    ret
14
15 main:
16    push    rbp
17    mov     rbp, rsp
18    sub     rsp, 16
19    mov     edx, 3
20    mov     esi, 2
21    mov     edi, 1
22    call    sum(int, int, int)
23    mov     DWORD PTR [rbp-4], eax
24    mov     eax, 0
```


A simple example

```
1  .LC0:
2      .string "Par01a"
3  check_password(char*):
4      push    rbp
5      mov     rbp, rsp
6      mov     QWORD PTR [rbp-8], rdi
7      cmp     QWORD PTR [rbp-8], OFFSET FLAT:.LC0
8      jne     .L2
9      mov     eax, 1
10     jmp     .L3
11 .L2:
12     mov     eax, 0
13 .L3:
14     pop     rbp
15     ret
16 main:
17     push    rbp
18     mov     rbp, rsp
19     mov     eax, 0
20     pop     rbp
21     ret
```

A simple example

The image shows a screenshot of the Compiler Explorer web application. The interface is split into two main panes. The left pane displays the C++ source code, and the right pane displays the generated assembly code for x86-64 gcc 13.2.

Compiler Explorer Interface:

- Top Bar:** Includes the Compiler Explorer logo, navigation links (Add..., More, Templates), a button to "Chat on our welcoming [Discord](#)", and links for Share, Policies, and Other.
- Left Pane (C++ source #1):** Contains the following C++ code:

```
1 // Type your code here, or load an example.
2 int check_password(char* psw) {
3     if (psw == "Par0la")
4     {
5         return 1;
6     }
7     else
8     return 0;
9 }
10
11 int main() {
12     return 0;
13 }
```
- Right Pane (x86-64 gcc 13.2):** Shows the assembly output for the provided C++ code. The assembly is color-coded to match the C++ code's logic.

```
1 .LC0:
2     .string "Par0la"
3 check_password(char*):
4     push    rbp
5     mov     rbp, rsp
6     mov     QWORD PTR [rbp-8], rdi
7     cmp     QWORD PTR [rbp-8], OFFSET FLAT:.LC0
8     jne     .L2
9     mov     eax, 1
10    jmp     .L3
11 .L2:
12     mov     eax, 0
13 .L3:
14     pop     rbp
15     ret
16 main:
17     push    rbp
18     mov     rbp, rsp
19     mov     eax, 0
20     pop     rbp
21     ret
```

A simple example

```
// Type your code here, or load an example.
int check_password(char* psw) {
    if (psw == "Par0la")
    {
        return 1;
    }
    else
        return 0;
}

int main() {
    return 0;
}
```

Resources

- Zenarmor. Reverse Engineering in Cybersecurity

<https://www.zenarmor.com/docs/network-security-tutorials/what-is-reverse-engineering>

- Reverse Engineering For Everyone!

<https://0xinfection.github.io/reversing/>

- Reverse Engineering Resources

<https://github.com/wtsxDev/reverse-engineering>

- C. Rusu – Reverse Engineering

<https://cs.unibuc.ro/~crusu/re/index.html>

- UPB. Security Summer School

<https://security.cs.pub.ro/summer-school/wiki/session/02>

- Stanford CS107. See Guide to x86-64 and CS107 x86-64 Reference Sheet

<https://web.stanford.edu/class/cs107/resources/>

- Intel® 64 and IA-32 Architectures Software Developer Manual.

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

- Felix Cloutier. x86 and amd64 instruction reference

<https://www.felixcloutier.com/x86/>